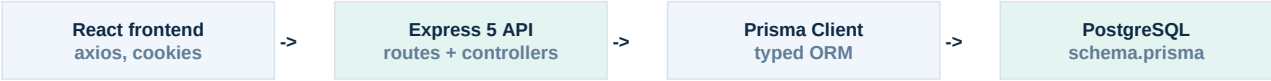


LaunchLens Backend - Architecture, Data Flow & Interview Guide

A code-grounded briefing based on the current **backend/** implementation. Prepared for interview preparation; no unverified components are presented as current behavior.



Architecture: frontend requests are routed through Express; protected endpoints use the cookie JWT middleware; persistence is centralized in one PrismaClient.

30-second explanation

LaunchLens is a campaign-link analytics backend. A signed-in owner creates projects and campaigns; each campaign receives a unique public slug. A public **/r/:publicSlug** request records a raw click event - visitor cookie, IP, browser, device and referrer - then sends a 302 redirect to the destination. The owner can retrieve aggregated analytics or request AI insights grounded in those aggregates.

Technology and actual backend shape

FILE	ROLE / INTERVIEW HOOK
Runtime	Node.js / TypeScript (ES2022, Node16 modules); Express 5
Data	PostgreSQL through Prisma 6; one config/prisma.ts client
Auth	bcrypt (10 salt rounds), jsonwebtoken, httpOnly cookie
Tracking	uuid/crypto UUID visitor cookie, ua-parser-js, request headers
AI	@google/genai SDK; Groq and Mistral REST fetch fallbacks; Zod 4 validation

Public and protected routes

Public: POST /api/auth/signup, POST /api/auth/login, GET /r/:publicSlug. **Cookie-protected:** GET /api/auth/me, POST /api/auth/logout, /api/dashboard, /api/projects, campaign CRUD, GET /api/campaigns/:id/analytics, and POST /api/campaigns/:id/insights.

Important distinction: this repo stores a nullable **country** field and returns country aggregation from the backend, but redirect tracking never sets it; the AI context builder excludes country, and the current CampaignAnalytics UI renders devices, referrers and browsers - not countries.

Backend file map - high value files

FILE	ROLE / INTERVIEW HOOK
src/app.ts	Mounts CORS, JSON/cookie parsing, all route modules, and final error handler.
controllers/redirect.controller.ts	Public click capture, UA parsing, ClickEvent create, then 302 redirect.
services/analytics.service.ts	Single source of campaign metrics; parallelizes seven analytics operations.
ai/aiInsight.service.ts	Ownership check -> analytics -> context/prompts -> provider -> JSON/Zod validation.

Database architecture and data ownership

Source of truth: `backend/prisma/schema.prisma` and migration `20260803132235_init/migration.sql`.

The datasource uses PostgreSQL with `DATABASE_URL` and `DIRECT_URL`.

Relationship diagram



All four foreign-key relationships use **onDelete: Cascade**. Delete a user and its projects/campaigns/clicks/conversions follow; delete a click and its optional one-to-one conversion follows.

Models that matter

FILE	ROLE / INTERVIEW HOOK
User	cuid primary key; required name/email/password; email is unique; createdAt default now, updatedAt automatic. Owns Project[].
Project	cuid primary key; required name and userId; optional description/website. Belongs to User and owns Campaign[]. Index: userId.
Campaign	cuid primary key; required name, destinationUrl, publicSlug, projectId; source/medium/campaignTag are nullable. publicSlug is unique; index: projectId. Owns ClickEvent[].
ClickEvent	cuid primary key; required campaignId and visitorId; optional ipAddress, country, browser, referrer; device enum defaults UNKNOWN; clickedAt defaults now. Indexes: campaignId, visitorId. Has optional Conversion.
Conversion	cuid primary key; required unique clickEventId and ConversionType; convertedAt default now. Schema supports one conversion per click, but no conversion controller/route was found in the current backend.

Enums and constraints

DeviceType: DESKTOP, MOBILE, TABLET, UNKNOWN. **ConversionType:** SIGNUP, DOWNLOAD, PURCHASE, DEMO_BOOKED. Database unique constraints exist for `users.email`, `campaigns.publicSlug` and `conversions.clickEventId`. There is **no database unique constraint** for project name per user; duplicate project names are prevented in controller queries, so this is application-level rather than database-level protection.

Slug creation: `generatePublicSlug.ts` slugifies the campaign name (lowercase, strict, trimmed) and appends a five-character `Math.random` base-36 suffix. The database unique constraint is the final collision safeguard; the controller does not retry on a collision.

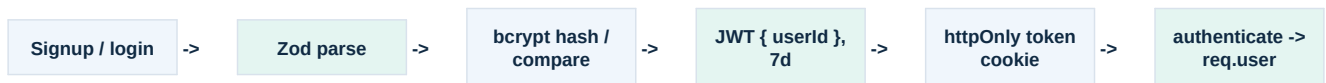
Why this schema fits the implementation

Project is the ownership boundary and Campaign is the public-link boundary. Click events stay raw enough for later aggregation, while the conversion relation reserves a one-click/one-conversion extension point. The FK indexes match the repository's recurring filters by `userId`, `projectId` and `campaignId`.

Authentication, authorization and tracking

Authentication identifies the requester. Authorization constrains queries to resources whose owning project belongs to that requester.

Flow 1 - authentication



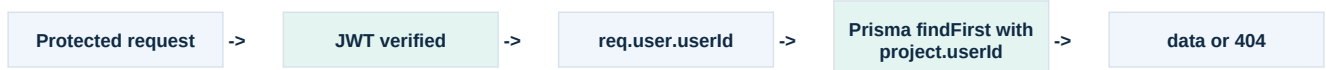
Signup validates name/email/password, checks unique email, hashes with bcrypt 10 rounds, and creates the user. It does not issue a token. Login validates, fetches by email, compares bcrypt hashes, signs the userId payload, and sets the cookie with sameSite=lax; secure is enabled only in production.

GET /api/auth/me is protected and selects id/name/email/createdAt. **POST /api/auth/logout** is protected and clears the token cookie. The middleware reads **req.cookies.token**, verifies via jsonwebtoken, and attaches **req.user = { userId }**; missing, invalid or expired tokens return 401.

src/middleware/auth.middleware.ts

```
const decoded = verifyToken(token) as { userId: string }; req.user = { userId: decoded.userId }; next();
```

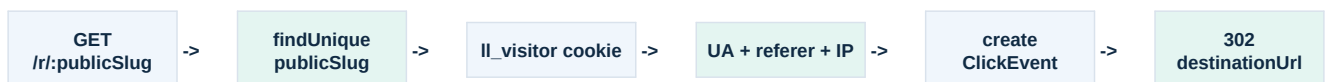
Ownership check pattern



Campaign analytics, AI insights, campaign update/delete, and project lookup all scope their Prisma lookup through the authenticated owner. Campaign creation/listing first scope the project by id + userId. This prevents User A from discovering User B's resource by ID: the scoped lookup returns not found.

Nuance: controllers use consistent ownership checks before reads/writes. Prisma parameterizes its generated operations, which avoids hand-built SQL injection in these paths. The generic error handler turns ZodError into 400; some controllers use asyncHandler to pass those errors, while other controllers catch and return generic 500 responses.

Flow 2 - public tracking link



Exact controller order: find campaign; get/create visitor ID; read User-Agent; detect device; parse browser; create the click; redirect. A missing campaign yields 404. A failed tracking operation yields 500 rather than redirecting.

User-Agent is an HTTP request header describing the browser/client. detectDevice.ts lowercases it and returns MOBILE if it contains 'mobile', TABLET if it contains 'tablet' or 'ipad', otherwise DESKTOP. ua-parser-js then attempts to obtain browser.name; absent or parsing failure becomes 'Unknown'. The controller reads **referer** (HTTP header spelling) and **req.ip** too. No geolocation lookup populates country.

Visitor IDs are a httpOnly, sameSite=lax, 365-day **ll_visitor** cookie made with crypto.randomUUID. They identify a browser profile, not reliably a person: different browsers or cleared cookies produce different visitor IDs.

Analytics: raw events to campaign metrics

GET /api/campaigns/:id/analytics first verifies campaign ownership in analytics.controller.ts, then delegates metric construction to services/analytics.service.ts.

Flow 3 - analytics and AI input



Promise.all runs independent campaign, summary, timeline, device, browser, country and referrer reads concurrently. That reduces latency compared with awaiting seven unrelated calls serially.

How the current queries calculate each metric

FILE	ROLE / INTERVIEW HOOK
Total clicks	prisma.clickEvent.count({ where: { campaignId } }) counts every event for this campaign.
Unique visitors	findMany with distinct: [visitorId] selects one row per visitor ID, then uses .length. It is browser-cookie uniqueness, not person identity.
Last click	findFirst ordered clickedAt: 'desc', selecting clickedAt; null if there is no event.
Devices / browsers	Fetches the relevant column and builds a TypeScript Map. Browser null becomes Unknown.
Referrers	Fetches referrer and Map-aggregates. A null referrer is labeled Direct; the code does not infer how that traffic arrived.
Timeline	Fetches clickedAt ascending, turns each timestamp into ISO YYYY-MM-DD, and Map-counts clicks per UTC date.
Countries	Fetches country and Map-aggregates null as Unknown. Returned by backend, but current tracking does not set country and it is excluded from AI context/UI.

src/services/analytics.service.ts

```
const [campaign, summary, timeline, devices, browsers, countries, referrers] = await Promise.all([getCampaign(id), getSummary(id), getTimelineAnalytics(id), getDeviceAnalytics(id), getBrowserAnalytics(id), getCountryAnalytics(id), getReferrerAnalytics(id)]);
```

Campaign list and dashboard

getCampaigns also calculates each campaign's total clicks and distinct visitor count, using Promise.all over the campaign list. getDashboard counts the signed-in user's projects, campaigns nested under those projects, and click events nested through campaign.project.userId; it also selects the five newest projects. These are separate from the single-campaign analytics service.

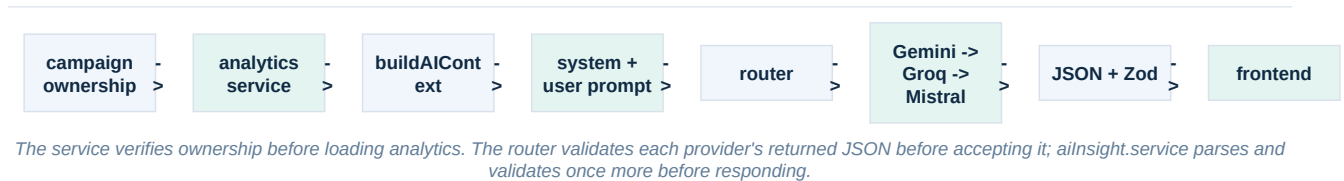
What the frontend actually consumes

CampaignAnalytics.tsx fetches analytics and renders summary cards, timeline, devices, top referrers and browsers. AllInsights.tsx POSTs to /campaigns/:id/insights and renders summary, insight cards and priority-tagged recommendations. It does not render country data, even though CountryAnalytics remains in the backend response type.

AI architecture, SDKs, fallback and validation

Every current file under backend/src/ai was inspected: aiInsight.service.ts, contextBuilder.ts, promptBuilder.ts, providerRouter.ts, testGemini.ts, and providers/{aiProvider,gemini,groq,mistral}.provider.ts.

AI flow



Files and provider implementation

FILE	ROLE / INTERVIEW HOOK
contextBuilder.ts	Turns CampaignAnalytics into campaignName, total/unique clicks, referrer/device/browser distributions with two-decimal percentages, and timeline. Countries are omitted.
promptBuilder.ts	Builds a restrictive system prompt plus a data-bearing user prompt. It permits only supplied data and requires JSON.
providers/aiProvider.ts	Shared interface: name and generateInsight(systemPrompt, userPrompt): Promise.
providers/gemini.provider.ts	Lazy-imports @google/genai, constructs GoogleGenAI({ apiKey }), and calls models.generateContent using gemini-2.5-flash, systemInstruction, user text, temperature 0.2; returns response.text.
providers/groq.provider.ts	Uses fetch to Groq's OpenAI-compatible chat/completions endpoint with llama-3.3-70b-versatile, 0.2 temperature and json_object response format.
providers/mistral.provider.ts	Uses fetch to Mistral chat/completions with mistral-small-latest, 0.2 temperature and json_object response format.
providerRouter.ts	Instantiates [Gemini, Groq, Mistral] in that exact order; parses and Zod-validates each result before return.
testGemini.ts	Standalone hand-authored test fixture that constructs sample analytics, invokes Gemini directly, and validates output; it is not a route test.

Fallback and prompt-engineering contract

Fallback: Gemini succeeds -> return immediately. If it throws - including missing key, API/empty response, invalid JSON, or schema mismatch - the router records the message and tries Groq; then Mistral. If all fail, it throws one controlled error containing the collected provider messages. The AI controller maps non-not-found errors to a 500 'Unable to generate AI insights.'

Grounding: the system prompt analyzes only total/unique clicks, referrers, devices, browsers and timeline. It prohibits inventing clicks, conversions, revenue, causes, channels or demographics; qualifies small data; treats Direct as a classification only; limits one <=2 sentence summary, <=4 insights, <=3 recommendations; requires JSON only. Structured JSON makes the frontend render predictable fields instead of parsing arbitrary prose.

Validation schema: summary string; insight type limited to traffic/source/device/browser/country/trend/data_quality with title/description/evidence strings; recommendations have high/medium/low priority with title/action. Schema validation checks type and presence, but does not independently fact-check an LLM's prose against the supplied numeric context.

Interview guide, security and limitations

Use these answers as code-faithful talking points. Where the repo has an extension point but not a route, say so clearly.

60-90 second technical explanation

LaunchLens is an Express/TypeScript API backed by Prisma/PostgreSQL. Users own projects, projects own campaigns, and campaigns own raw click events. Cookie JWT authentication gives the backend a `userId`; each project/campaign operation scopes its Prisma lookup through that owner. A public redirect URL looks up the unique campaign slug, gets a persistent browser visitor cookie, captures headers, stores a `ClickEvent`, and sends a 302 redirect. The analytics service counts and aggregates raw events in parallel. The AI path uses those existing analytics, builds a restricted context and prompt, then attempts Gemini, Groq and Mistral in sequence. Output must be JSON and pass Zod before the frontend receives it.

Two-minute click -> analytics -> AI deep dive

A visitor opens `/r/:publicSlug`. `redirectToCampaign` finds the campaign by unique `publicSlug`. It reuses or creates `ll_visitor`, reads `User-Agent`, classifies device with simple string checks, derives `browser.name` through `ua-parser-js` where possible, reads `referer` and `req.ip`, and inserts `ClickEvent`. It then returns a 302 to `destinationUrl`. When the owner opens analytics, the controller proves `campaign.project.userId` matches the JWT `userId`. `getCampaignAnalytics` uses Prisma `count/distinct/findFirst` and `Map` aggregation to return metrics. If the owner asks for insights, the AI service repeats the ownership check, reuses analytics, omits country from the AI context, applies grounding instructions, sends the prompts through the ordered provider router, and only returns parsed Zod-valid JSON.

Security and deployment checklist

FILE	ROLE / INTERVIEW HOOK
JWT / cookies	7-day JWT contains <code>userId</code> only; <code>httpOnly</code> token cookie, <code>sameSite=lax</code> , secure only when <code>NODE_ENV=production</code> .
Passwords	<code>bcrypt</code> hash/compare with 10 salt rounds; password is never selected in <code>me()</code> response.
Access control	Scoped ownership lookups by <code>project.userId</code> before protected resource access or mutation.
Input and errors	Zod validates <code>auth/project/campaign</code> and AI output; error middleware formats Zod errors. Coverage is not uniform across controllers.
Secrets	Env names used: <code>PORT</code> , <code>DATABASE_URL</code> , <code>DIRECT_URL</code> , <code>JWT_SECRET</code> , <code>APP_URL</code> , <code>GEMINI_API_KEY</code> , <code>GROQ_API_KEY</code> , <code>MISTRAL_API_KEY</code> . Never commit real <code>.env/API</code> keys.
CORS	Configured for <code>http://localhost:5173</code> with credentials; deployment must intentionally update the allowed origin.

15 realistic interviewer questions - concise answer / deeper follow-up

Why PostgreSQL + Prisma?	Relational ownership fits <code>User -> Project -> Campaign -> ClickEvent</code> ; Prisma gives typed query APIs and migrations.	Deeper: FKs/cascades and indexes live in schema/migration.
How do you stop cross-user access?	JWT yields <code>userId</code> ; each lookup includes <code>userId</code> through the project relation.	Deeper: A non-owned ID returns 404, not data.
What happens on a link click?	Find slug, read visitor/headers, save <code>ClickEvent</code> , 302 redirect.	Deeper: Tracking failure returns 500 before redirect.
What is a ClickEvent?	A raw campaign interaction with <code>visitorId</code> , metadata and <code>clickedAt</code> .	Deeper: It is the source for every aggregate.
How are unique visitors calculated?	Distinct <code>visitorId</code> rows, then length.	Deeper: It approximates a browser, not a human.
How detect browser/device?	<code>ua-parser-js</code> reads browser name; simple UA checks classify device.	Deeper: Unknown browser is allowed; device fallback is desktop.
What is User-Agent?	A request header that identifies client software/device hints.	Deeper: It can be absent/spoofed, so detection is best-effort.
Why Promise.all?	Seven independent analytics reads run concurrently.	Deeper: It lowers response latency vs serial awaits.
How does AI stay grounded?	Prompt limits data and output plus Zod schema validation.	Deeper: Schema validates shape, not factual truth; provider text still needs caution.
Why provider abstraction?	One interface lets the router swap providers without changing AI service.	Deeper: Order is Gemini, Groq, then Mistral.
If Gemini fails?	Router catches, validates failures too, then tries Groq and Mistral.	Deeper: All failures become a controlled service error and 500.
What does GoogleGenAI do?	It is the <code>@google/genai</code> client used to call <code>generateContent</code> .	Deeper: The app uses <code>gemini-2.5-flash</code> with <code>systemInstruction</code> and temperature <code>.2</code> .
What is temperature here?	A model sampling control; the code sets <code>.2</code> for all providers.	Deeper: Lower variance helps a structured analytic response.
Are conversions implemented?	The schema has <code>Conversion</code> and enum types.	Deeper: No conversion endpoint/controller was found.
What are MVP limits?	No country population, no conversion flow, no AI factual verifier, and UA/cookie limits.	Deeper: CORS is currently localhost-specific; AI relies on external providers.